

A modelling engine for UK welfare benefit entitlement and household income

Daniel Whitston

MSc Computer Science project report

Department of Computer Science and Information Systems,

Birkbeck College, University of London

2022

This report is substantially the result of my own work, expressed in my own words, except where explicitly indicated in the text. I have read and understood the sections on plagiarism in the Programme Handbook and the College web site. I give my permission for it to be submitted to the JISC Plagiarism Detection Service.

The report may be freely copied and distributed provided the source is explicitly acknowledged.

Contents page

Contents page	2
Abstract	5
Terminology	6
Background	7
The problem space	7
The opportunity space	8
Open, verifiable modelling of UK benefits	8
Integration of an SMT solver	8
Aims and objectives	9
Implementation	10
Objective 1: Create a sample set of benefit rules and test cases	10
Limits to the model	10
Location	10
Dates	10
Payments vs eligibility	11
Ruleset 1 - When you come under the universal credit system	11
Ruleset 2 - Are you eligible for Universal Credit?	12
Ruleset 3 - How much Universal Credit do you get?	12
Ruleset 4 - How much is your maximum amount of UC?	13
Ruleset 5 - What is the earnings taper?	13
Ruleset 6 - What is unearned income for UC?	13
Ruleset 7 - The standard UC allowance	13
Ruleset 8 - An allowance for children under UC	13
Ruleset 9 - An element for adults with limited capability for work or work-related activity due to illness under UC	14
Ruleset 10 - A carer element if one of the recipients cares for a severely disabled person under UC	15
Ruleset 11 - An allowance for childcare under UC	15
Ruleset 12 - A housing costs element under UC	16
Private tenant	16
Tenant of a registered social landlord	17
Owner-occupier paying mandatory service charges	17
Test case 1 - UC maximum amount, with Gail and Joe	17
Objective 2: Define a language for expressing benefit rules	18
Structure and flow	18
Statements	18
Test case statements	19
Whitespace and newlines	19
Comments	19

List	19
Expression	20
Term	20
Datatypes	21
Integer	21
Money	21
Percent	22
Date	22
Boolean	22
Enum	22
Fraction - not implemented	22
List or array - not implemented	23
Operators	23
Assignment: =	23
Equality: ==	23
Less-than and greater-than: < and >	23
Less-than-or-equal-to and greater-than-or-equal-to: <= and >=	24
Addition: +	24
Zero-bounded subtraction: ~-	24
Multiplication: *	24
Division: /	24
Minimum: min	24
Conditional: if then else	25
Conditional: case when when - not implemented	25
Maximum: max() - not implemented	25
Discussion points	25
Distinguishing eligibility and amount	25
Dealing with multiple instances of an 'object'	26
Objective 3: Create a parser to parse the benefit rule language	26
Operator precedence	26
Objective 4: Create a runner that takes language code and sample data	27
Listener vs visitor patterns	27
Z3 representation of Ben datatypes	28
Integer	28
Money	28
Percent	29
Date	29
Boolean	29
Enum	29
The main function	30
Using the runner	30
Evaluation	30
The capabilities that have been achieved	31
The limitations of what's been achieved	31

Deviations from the original plan	32
What would have improved the process	33
Opportunities for further development and exploration	33
Conclusion	33
References and Credits	35
Bibliography	35
Appendices	38
Appendix 1: Sample benefit rules, written as a Ben file	38
Appendix 2: Sample test cases, as Ben files	45
Appendix 3: Ben lexer and parser, written as an ANTLR4 grammar file	46
Appendix 4: Python and Z3Py runner script	52

Abstract

Modelling of benefit rules is used at all levels of benefit policy and delivery in the UK. This includes supporting individual claimants, understanding local needs to deliver effective interventions, and modelling and policy-making at the national level.

The models used by different organisations to perform these activities are largely proprietary, and cannot be externally audited or verified. Additionally, none of them integrate with automated theorem provers, which would both extend the range of solvable inputs, and allow for automated investigation of hypotheses about the effects of benefit rules.

This project's intent was to develop a proof of concept system, consisting of:

1. A sample set of benefit rules and test cases, for use in developing and evaluating
2. A domain-specific language for benefit rules called Ben, consisting of a narrative language definition and a formal grammar, plus a conversion of the sample rules and test cases to Ben
3. A lexer and parser for Ben, which successfully parse the sample Ben rules
4. A runner, which converts the parsed rules and test cases to an SMT solver-compatible form, then uses the SMT solver to determine satisfiability and provides solutions if available.

This report discusses the development decisions and reasoning behind the language, rules, and test cases. While the project was successful as a proof of concept, the resulting model has many limitations that would need to be addressed for production readiness. These limitations are also discussed, as are opportunities for further development and real-world use of the model.

Terminology

Benefit or **benefits**, **tax-benefit**, **welfare** and **social security** are used interchangeably throughout this report. Benefit calculations implicitly require the modelling of individual and household taxes, so tax modelling is usually included in any benefit calculation.

Model, **calculator**, or **simulation** are used interchangeably to refer to the software under discussion.

UK welfare benefits refers to all three systems operated in the UK: England and Wales, Northern Ireland, and Scotland. Although administered separately, the differences are not at present great enough to require separate modelling as devolved governments work to maintain parity on devolved benefits (Mackley, 2022), so the same models can be applied across all three systems with appropriate adjustments.

Claimant refers to the person making a benefit claim, i.e. the person whose situation is being modelled by the benefit rules. Universal Credit requires a joint claim to be made for a couple who live together, so many of the rules refer to the *claimant(s)*, the *claimant or couple*, or the *partner*, to distinguish between the different individuals in such a situation.

Universal Credit (UC) is a new set of benefit rules that has gradually replaced several 'legacy' benefits over the past decade. During the roll-out of the new system, modelling of benefit eligibility has had to cover both systems, along with the rules used to distinguish whether a claimant comes 'under' the legacy system or UC.

ANTLR4 is a **parser generator** that takes a grammar file containing lexer and parser definitions written in a variant of **EBNF** (extended Backus-Naur form), and outputs source code that converts a program following the grammar to an **AST** (abstract syntax tree), which can then be walked in order to interpret or convert the tree to another form.

Z3Py is an API for **Z3**, providing a compatibility and interoperability layer with Python. Z3, in turn, is a theorem prover (or **SMT solver**) that allows theorems to be expressed and automatically checked for satisfiability and possible solutions. It builds on the **SMT-LIB2** language for direct interaction, but can also be used via Z3Py.

Background

The problem space

The benefit system disburses money (and, occasionally, other goods or services) to individuals or households in order to improve their standard of living. In the UK, these systems operate following rules defined in government legislation, with further guidance provided by statutory instruments, ancillary publications and departmental notes, and case law (The National Archives, 2018). The requirement to take account of precedent from case law marks the UK out as different to the civil law system practised in France and other countries (Husa, 2016), and is worth remembering when comparing models across different systems.

The rules that make up the benefit system are modelled by many different parties, for different purposes, and at different scales and depths. There is no single 'official' model that acts as a reference; the official system is laid out in the aforementioned legislation and notes. This means that each organisation that wishes to understand how the system works and understand its impact on individuals or society has to distil the various rules into their own model.

Individuals often use benefit calculators to determine benefit eligibility, and to predict what the results of e.g. moving into work might be for their future income. The three main calculators are provided by entitledto, Turn2Us, and Policy in Practice (UK Government, 2022). These are all proprietary, closed source solutions, and their code and models are not open to verification or update by outside parties.

On a wider scale, household finances are modelled by local authorities, registered social landlords and other social purpose organisations, using so-called 'administrative data' (Administrative Data Research UK, 2022) which provides partial insight into household situations but misses key pieces of data. These may use either scaled-up versions of the individual benefit calculators or so-called microsimulation models. Either way, they use simple heuristics and broad-brush assumptions to account for missing data.

Local authorities and national government departments operate proprietary systems that determine and process tax and benefits, using a combination of automated rules-based reasoning, along with individual decision making by civil servants that is then implemented by the automated systems. These operate at the individual level and are therefore extremely detailed - they represent the territory rather than the map. Some of these systems are run by outsourcing companies that have in turn developed their own proprietary benefits models.

Finally, research and modelling by the research and policy-making community uses microsimulation models to understand policy impacts and economic changes, applying headline rulesets to statistical data such as the Family Resources Survey (Richiardi, Collardo and Popova, 2021). These include UKMOD, TAXBEN and the IPPR tax-benefit model. UKMOD's model is partially open, but is not intended to apply a full set of rules or to make use of actual household inputs or administrative data.

Taken together, the benefits system is represented in dozens if not hundreds of separate rulesets, with no public oversight, no sharing of rules, no sharing of effort to reduce duplication, and (to the author's knowledge, as former CTO of one of the benefit calculator providers) no use of formal verification or theorem proving techniques to provide assurance of correct and complete behaviour.

The opportunity space

All of this is in stark contrast to a near neighbour, the French tax system. The government website has both simple and detailed tax simulators (Directorate of Legal and Administrative Information, 2022). Per Merigoux, Monat and Protzenko (2021), an open source compiler called Mlang is being used by the French government to calculate taxes in their production system. More exciting for the longer term, Delaët, Merigoux and Fromherz (2022) detail a language for representing fiscal law. The language is called Catala and is also available on GitHub (Merigoux et al., 2022), with examples including partial modelling of both French and US tax law. When law is described in the language, in addition to compiling to Python for production use and to PDF for lawyer readability, it becomes provably correct through compilation to F* and running with Z3.

The examples above inform this project's exploration of two main areas:

1. Open, verifiable modelling of UK benefits for use by individuals and organisations
2. Integration of an SMT solver to add flexibility to the model's usage, and to enable investigation and determination of hypotheses about the UK benefit rules

Open, verifiable modelling of UK benefits

The duplication of effort, opacity of systems, and proprietary nature of most models adds friction and uncertainty to all benefit modelling activities. The direct consequences of individual decisions made with bad data are incalculable, as are the indirect consequences with respect to policy decisions.

By contrast, an open, communally maintained benefits model would reduce duplication, reduce cost, increase transparency and verifiability of the model's rules, and vastly reduce the risk of users obtaining incorrect or outdated results.

A domain-specific language would be particularly useful in building and maintaining such a system. In part because it would be easier for domain specialists to read and understand, and in part because it would provide a constrained structure that directed the expression of the model's rules down a consistent and more easily understood path.

Integration of an SMT solver

By itself, an open model developed using a DSL would be useful. However, it would still only duplicate the functionality of existing models. By integrating an SMT solver into the language parser, it would be possible to explore the benefits system in entirely new ways:

- A benefit calculation normally requires hundreds of pieces of information to reach a final determination, with no recourse if e.g. the date of birth of a housemate's child is unknown. Additionally, the direction of information always goes from inputs to

outputs. An SMT solver could be used to try and determine end results through a mix of known values, which in reality often includes a mix of household circumstances and known eligibility.

- A more ambitious use would be to determine the most efficient input tree, that is, what order to ask questions in such that a user's total number of inputs before reaching a desired level of result accuracy is minimised.
- Likewise, modelling households with administrative data normally requires the use of assumed values for data that's missing, or of crude back-modelling from known outputs. Filling in the gaps or determining possible ranges of values would lead to more accurate results, with less manual assumption-making.
- The solver could be used to investigate the correctness and completeness of the model itself, or even the wider benefits system. For example, a stated aim of the benefit system is that earning more money should never result in lower overall income. An SMT solver could be used to search for circumstances in which that rule did not hold, which could then drive changes to the benefit rules.

Aims and objectives

The project aims were to explore the two opportunity spaces (modelling of benefits via a DSL, and integration of an SMT solver), to develop understanding of their potential, and to provide a base for future development in the field of benefit modelling.

To achieve these, the objectives were to:

1. Create a sample set of benefit rules and test cases
2. Define a language for expressing benefit rules
3. Create a parser to parse the benefit rule language
4. Create a runner that takes language code and sample data

These objectives were intended to be tackled in order as individual development stages, building on each other to produce a coherent toolkit and process for modelling benefits. The process was to follow a test-driven approach, with the use of test cases for each stage, in some cases derived from the test cases developed in the previous stages. The code was also intended to be clean, well-structured and in adherence with modern development practices.

Implementation

For each objective, there follows an overview of the work carried out, and the issues and solutions found.

Objective 1: Create a sample set of benefit rules and test cases

The source used for benefit rules is The Welfare Benefits and Tax Credits Handbook (Child Poverty Action Group, 2018). The rules are expressed in the author's own language, with references, to mitigate any risk of questions around copyright and plagiarism.

The original plan was to model a random sample of rules from the book, but this would have had no real modelling value when it comes to understanding the behaviour of a more complex system of rules. Thus, the model starts with one regulation and spread outwards to cover an interconnected set of dependent rules. This will enable the modelling of a system of rules instead of just running them in isolation, which will make it easier to test the behaviour of the system and the SMT solver with more complex models. The SMT solver would be of no benefit in 'solving' individual rules.

Limits to the model

The main limitation of the model is that we are modelling a subset of the overall benefits rules, to establish household benefits eligibility at the date of first claim, where the first claim date is in the period of validity of the handbook, i.e. 5/3/2018 to 4/3/2019. For now, the subset is Universal Credit, in full service areas. These limitations are explained further in the descriptions of benefits rules.

Location

Per Child Poverty Action Group (2018, pp.3-4), the handbook's rules apply to England and Wales. Scotland has broadly the same legislation but has the power to diverge. Northern Ireland has separate legislation on benefits and tax credits, although the systems are similar in practice. Beyond this, where a claimant lives has further impact on several aspects of benefits - there are local taxes and benefits such as council tax and discretionary housing payments; the council tax also varies according to the designated property band of the claimant's home and their household status.

Dates

In common with all rules being discussed, the rules have changed in the years since the handbook was published. In this case the change is particularly acute: no new claims can be made for the benefits classed as legacy benefits, since all new claims are for Universal Credit. This does not affect the correctness of the rules being modelled, *for the period in which those rules were in effect*. All benefit rules have the same issue of time dependence. Not only the conditions of eligibility but also the payment amounts and even the existence of rules changes over time. Not only do claimants' circumstances change, but people get older, which changes their eligibility in and of itself.

Since the handbook we're using as a basis for our modelling only applies to a limited timespan, we will limit our modelling to claims with a date of new claim falling within the period of validity. The handbook's acknowledgements (Child Poverty Action Group, 2018, p.iv) state that the law covered in the book was correct on 5/3/2018, so we shall set the limit for new claim dates to between 5/3/2018 and 4/3/2019, for now. Likewise, we will not model payment dates outside of those lower and upper limits.

Payments vs eligibility

There are situations in which someone's eligibility for a benefit is not established until after the point where that benefit should have been paid. Sometimes this results in payment being backdated, i.e. paid at a later point. In other circumstances, the payment may not be made up at a later point. Either way, we're looking to model benefit eligibility rather than the payment of eligible and claimed benefits, which has many rules of its own and relies implicitly on the history of payments made to a claimant.

Ruleset 1 - When you come under the universal credit system

Source: Child Poverty Action Group (2018, pp.20-22)

The handbook was published during transition between two benefit systems: moving from a legacy system consisting of several sets of benefits and tax credits, to Universal Credit (UC). Whether claimants would be assessed under the legacy system or UC was dependent on date of initial claim, the area their home address was in, and several other conditions.

A claimant comes under UC if either:

1. Their date of claim for UC is after the rollout date for the Jobcentre area in which they're claiming, as listed in the PDF files at Department for Work and Pensions (2016). The rollout areas are often known as 'full service' areas. We know from the linked URL that rollout completed in December 2018, in accordance with the target date stated on p22 of the handbook.
2. Their date of claim for UC is before 1/1/2018, and after April 2016, and the Jobcentre area in which they're claiming is a UC gateway area, and they meet a long list of gateway conditions that are fully detailed in Chapter 2 of the 2017/18 edition of the handbook. An immediate consequence of the date limitations discussed earlier on is that our model completely excludes UC gateway claimants, since we're taking new claims from 5/3/2018.

There is an exception to the rule for full service areas: claimants with three or more children, and who meet certain other conditions, should continue to claim legacy benefits for claims between 6/04/2017 and 31/01/2019 (i.e. during the modelled period).

As with other rules, there are inputs to this decision which are themselves subject to modelling. In this model, those are:

- The date of claim, which is a date input between 5/3/2018 and 4/3/2019.
- The date at which their Jobcentre area becomes a 'full service' area, which is a date between November 2015 and December 2018
- Whether a claimant meets the three-or-more-children-based exception criteria for UC rollout.

Ruleset 2 - Are you eligible for Universal Credit?

Source: Child Poverty Action Group (2018, pp.31-33)

There are several requirements which must *all* be true for a claimant to be eligible for UC. UC normally treats a couple as a single unit for claim purposes, i.e. they make a single claim and are paid jointly. For a couple to be eligible to claim UC normally, all requirements must be true for both partners.

The basic requirements to claim UC are that the claimant(s) are:

- coming under UC, i.e. that rule 1 is true
- aged 18 or over, or aged 16-17 and meet special young person UC eligibility criteria
- aged below the qualifying age for Pension Credit (PC)
- not receiving education unless you are exempted from this requirement
- in habitual residence in Great Britain and have the right to reside in Great Britain
- either physically in or can be treated as being in Great Britain
- not a person subject to immigration control
- in acceptance of the claimant commitment

It is possible for one member of a couple to claim as a single person if they meet the basic requirements themselves, but their partner fails to meet basic requirements in a certain, limited set of ways. In this situation, the claim works as if the claimant is a single person, *except* that both partners' income, savings and capital are still included in calculations.

The financial requirements to claim UC are that claimant(s) do not have:

- too high an income
- savings and capital above £16000

The income ceiling ('too high an income') is not a fixed number - instead it refers to an income which is high enough that no UC would be paid, due to the reduction in UC payments that results from income. This introduces a dependency between this ruleset and rule 3, the expected amount.

Ruleset 3 - How much Universal Credit do you get?

Source: Child Poverty Action Group (2018, pp.34-38)

Much of the detail on calculation of amounts in this section of the handbook simply repeats information from elsewhere in less detail, leaving a relatively simple ruleset. Within an assessment period, per steps four and five on Child Poverty Action Group (2018, p.38):

Universal Credit entitlement = maximum amount - earnings taper - unearned income

The terms are defined in rules 4, 5 and 6, respectively. There is also a set of rules for something called Transitional Protection, but this only applies after July 2019, so is not a concern for our model. Where relevant, the time period used for any value calculated over time in UC (e.g. rent being paid, childcare cost) is £ per month.

Ruleset 4 - How much is your maximum amount of UC?

Source: Child Poverty Action Group (2018, p.59)

The UC maximum amount is the sum of:

- The standard UC allowance for the person or couple that are claiming (rule 7)
- An allowance for children (rule 8)
- An element for adults with limited capability for work or work-related activity due to illness (rule 9)
- A carer element if one of the recipients cares for a severely disabled person (rule 10)
- An allowance for childcare (rule 11)
- A housing costs element (rule 12)

Ruleset 5 - What is the earnings taper?

Source: Child Poverty Action Group (2018, pp.36-37)

$\text{Earnings taper} = \text{maximum of } 0 \text{ or } (\text{net earnings} - \text{work allowance}) \times 0.63$

where

net earnings is the monthly earned income after tax, national insurance, and pension contributions. If a couple are claiming, the sum of the net earnings of both claimants is used.

work allowance is counted if the claimant(s) are responsible for a child or have limited capacity to work. If counted, then the value is £198 if there is a housing costs element (i.e. rule 12 has a greater-than-zero result) or £409 if there is no housing costs element. Otherwise, it's £0.

Ruleset 6 - What is unearned income for UC?

Source: Child Poverty Action Group (2018, p.37)

This can include income from a large range of unearned sources, including certain other benefits, income from capital including rental income, and so on. No taper is applied to unearned income, so the full value is counted against the UC maximum amount.

Ruleset 7 - The standard UC allowance

Source: Child Poverty Action Group (2018, pp.59-60)

The standard UC allowance is £251.77 for a single claimant aged under 25, £317.82 for a single claimant aged 25 or over, £395.20 for a claiming couple both aged under 25, £498.89 for a claiming couple where at least one person is aged 25 or over.

Ruleset 8 - An allowance for children under UC

Source: Child Poverty Action Group (2018, pp.60-61)

A claimant or their partner are responsible for a child if the child 'normally' lives with them. A child counts toward the allowance if they are (i) under 16 years old, or (ii) if they are above or equal to 16 years old, below 20 years old, and meet the criteria for a 'qualifying young person' per Child Poverty Action Group (2018, p.56).

If the eldest qualifying child (i.e. the one with the earliest date of birth) was born before 06/04/2017, then the allowance element for that child is £277.08 per month (i.e. per assessment period). Otherwise, the allowance element for them is £231.67.

If there is a second qualifying child, the allowance element for them is £231.67 per month (i.e. per assessment period).

There is a two-child limit for new UC claims, which adds considerable complexity:

- Claimants / couples with more than two qualifying children at time of claim do not come under UC and should claim legacy benefits instead, for new claims starting between 06/04/2017 and 01/02/2019. This requirement is described in Ruleset 1, and would *normally* mean that new claimants with more than 2 eligible children did not come under UC, and so did not need to be modelled.
- However, there are several exceptions to the two-child limit for special circumstances such as multiple births (i.e. twins or triplets). It would take more investigation to determine whether these would take effect on claimants that come under UC, or whether any such claims would be diverted to legacy benefits. For example, a claimant with two eligible children claims early in the period under modelling, comes under UC and is successful in their claim, then has another child. Do they remain under UC and hit the two-child limit, or are they switched to legacy benefits?
- At the time of publication of the handbook, there was also a legal challenge under way against the two-child limit, which would potentially have required backdated eligibility and benefit payment if it had succeeded. It's unclear how this kind of retrospective alteration to eligibility could be modelled.

For the sake of simplicity, it is assumed for now that there can be no UC claimants / couples with three or more eligible children.

Separately to the normal child elements, a claimant receives a disabled child addition for each child in their or their partner's care. This is based on assessment of the child's health and disability status. Each child awarded a lower rate addition contributes an element worth £126.11 per assessment period. Each child awarded a higher rate addition contributes an element worth £383.86 per assessment period. The two-child limit does not affect these values, although, again, we are limiting consideration of claims involving more than two children for this model.

The total allowance for children under UC is the sum of the allowance elements.

Ruleset 9 - An element for adults with limited capability for work or work-related activity due to illness under UC

Source: Child Poverty Action Group (2018, pp.64-67)

There are two parts to this element:

1. Limited capability for work - This does not exist for new UC claims from 03/04/2017, but is still possible to receive for ongoing claims from before that date, or if covered by a long list of related edge cases to do with existing claims. It is assumed that none of these apply to the model under development, as it only deals with new claims in the period under modelling.
2. Limited capability for work-related activity - This is a more strictly defined status than limited capability for work, and is normally awarded following a Work Capability Assessment (WCA) that may include medical examination at an approved assessment centre. The criteria for deciding on an award are complex and ambiguous, with frequent appeals and reversals of initial assessment decisions.

Dealing purely with limited capability for work-related activity, if a claimant or their partner are found to have limited capability for work-related activity, then they receive £328.32 per month (i.e. per assessment period). There is no extra award if both members of a couple are assessed as having limited capability for work-related activity.

There is complexity to the timing of eligibility - a newly qualifying claimant should only have the award counted from the first assessment period beginning more than three months from the date of first providing medical evidence of a health condition that leads to Work Capability Assessment. So a claimant provides evidence such as a medical certificate from their GP, this triggers the Work Capability Assessment process, and eligibility starts on the first assessment period that begins more than three months after that date. If the WCA process takes longer than that, then payment will be backdated following award.

Ruleset 10 - A carer element if one of the recipients cares for a severely disabled person under UC

Source: Child Poverty Action Group (2018, pp.67-68)

If a claimant or their partner provides unpaid care for someone, for at least 35 hours per week, they are eligible for the carer element. Along with a range of criteria to establish whether someone actually is a carer in this definition, there are several caveats:

- The element for adults with limited capability for work-related activity takes precedence. If a carer is also eligible for the work-related activity element, they get that and not the carer element.
- If a claiming couple both care for the same person, they are only eligible for the element once. If they meet carer conditions for a separate person each, they are eligible for two carer elements.

The carer element is £156.45 per month (i.e. per assessment period), or twice that if both partners are caring for different people.

Ruleset 11 - An allowance for childcare under UC

Source: Child Poverty Action Group (2018, pp.68-70)

If a claimant pays for childcare in order to carry out paid work, then they are eligible for repayment of 85% of childcare costs, up to a maximum of £646.35 per month (i.e. per assessment period) for one child, or £1108.04 per month for two or more children. This comes with conditions, including:

- There is no minimum number of hours, but it must be declared, paid work.
- If a couple are claiming, then either both partners must be in work, or the non-working person must be assessed as incapable of work-related activity or a full-time carer.
- The childcare has to be by a recognised provider.
- The child has to be under 16, or aged 16 and the childcare period is before the 1st September after their 16th birthday.

There are further conditions, but they do not affect the basic rules. There is also a help scheme called tax-free childcare payments, but claiming it removes eligibility for UC.

Ruleset 12 - A housing costs element under UC

Source: Child Poverty Action Group (2018, pp.72-106)

The rules surrounding housing costs eligibility are complex, and would be unrewarding to include in our current model, since they largely consist of a long array of conditions which would exclude a claimant from eligibility, and our modelling interest is primarily in the interaction of rules. Accordingly, a basic model is described in the following sections, which assumes those conditions are satisfied and a claimant is eligible.

A claimant / claimant couple may have the following statuses:

- Ineligible for housing costs or not claiming them
- Private tenant
- Tenant of a registered social landlord
- Owner-occupier paying mandatory service charges

A claimant / couple who do not or cannot claim for housing costs get no contribution from this to their UC maximum amount. Some of the statuses that lead to ineligibility introduce requirements on other rules, and these are not modelled. For example an 18-22 year old claimant is usually ineligible for a housing element, and not modelling this means that it's possible for a claim modelled in our subset of the rules to break rules that are present in the full benefit system.

If any of the remaining three statuses return a value that is 0 or less than 0, then this has the same effect as being ineligible for housing costs.

Private tenant

Source: Child Poverty Action Group (2018, pp.94-95)

The housing costs element for a private tenant is calculated as:

```
Private tenant housing costs element = minimum of ('core rent' and 'cap rent') - non-dependent deduction
```

where

$\text{core rent} = \text{liable rent} \times \text{number of relevant family members} / \text{total number of liable people}$ (Child Poverty Action Group, 2018, p.96)

$\text{cap rent} = \text{local housing allowance for property with your allowed number of bedrooms}$ (Child Poverty Action Group, 2018, p.97)

$\text{non-dependent deduction} = £72.16 \text{ for each non-dependent who is resident and in your extended benefit unit}$ (Child Poverty Action Group, 2018, p.93)

The local housing allowance is an amount that varies by Broad Rental Market Area, i.e. house location, and by allowed number of bedrooms, i.e. the number of bedrooms deemed appropriate for the occupants of the house.

Tenant of a registered social landlord

Source: Child Poverty Action Group (2018, pp.98-99)

The calculation for a social tenancy is:

$\text{Social tenant housing costs element} = \text{rent} - \text{bedroom tax}$

where bedroom tax is 0 if you have the allowed number of bedrooms for your family size, rent x 0.14 if you have one bedroom more than allowed, and rent x 0.25 if you have two or more bedrooms above your allowance.

Owner-occupier paying mandatory service charges

Source: Child Poverty Action Group (2018, pp.101-104)

While e.g. mortgage repayments are not covered by UC, unavoidable service charges excluding charges for utilities such as water and electricity can be covered. However, there is a nine month qualifying period, so UC claimants who are owner-occupiers of their home will not receive any housing costs element until they have been claiming UC for nine months. There are several caveats to this rule, as always.

Test case 1 - UC maximum amount, with Gail and Joe

Source: Child Poverty Action Group (2018, p.36)

Situation: Gail and Joe are a couple, both over 25 years old and under pension age. Their UC date of claim is May 2018. Their two children are both born after 6th April 2017. Gail works. Joe has limited capability for work and work-related activity through an existing finding, so does not have a waiting period to become eligible. They have no eligible housing or childcare costs, and no unearned income.

Inference: Their UC maximum amount is £1290.55.

Objective 2: Define a language for expressing benefit rules

Now that a set of sample benefit rules have been characterised, it is possible to identify the language elements that are needed to represent the rules. The name of the language is Ben, and the aim is to keep it simple. This simplicity is partly because the rules should not require too much complexity to capture, and partly because a simpler language will be easier to express as a grammar and convert to Z3Py. As long as the structure is reasonably well-formed, it should be possible to extend later without having to significantly rework any Ben code that has been written.

The rules and the test cases from Objective 1, expressed as Ben source code, can be found in [Appendix 1](#) and [Appendix 2](#), respectively.

Structure and flow

The benefit rules as described are a set of statements relating outputs (a claimant or couple's benefit entitlement) to inputs (their household circumstances, employment etc). While the rules described the results as deriving from the inputs, part of the point of the model is to take a combination of known and unknown inputs and results and use the relations to find the missing values throughout the model.

Taking a bottom-up approach, we can identify the individual pieces of data and logical operators required to capture the sample benefit rules, and to work with them in the context of an SMT solver. This plus some function definitions, and program flow methods, should be enough to model the sample benefit rules.

Statements

In a set of rules that define benefit conditions, we have the following kinds of statement:

- Comment - annotate the rules defined in code. No impact on modelling, but useful for documentation.
- Function creation - declare a rule that a variable is equal to some expression. This is the basic unit for defining each rule that governs how characteristics of a household relate to each other. For example, `Money uc_housing_private_core_rent = { (rent * number_of_relevant_family_occupants) / number_of_liable_occupants }`.
- Declaration - define a variable for use in the expressions that make up the set of assignments. By declaring them in advance, we have explicit knowledge of their type, or `sort` as it's known in SMT-LIB2.

Enums have slightly different grammar and an additional declaration step compared with other datatypes. It's necessary to first define the set of values within a named enum, then declare a variable that takes a value from the enum now that it's been defined.

Test case statements

To test or interact with a model derived from a set of benefit rules, we need two further types of statement:

- Assignment of explicit value to variables - match the variables to a household's situation.
- Assertion of an 'unknown' variable's value - use an SMT solver to confirm that the known variables, when applied to the rules given, can be used to confirm the value of the 'unknown' variable.

The SMT solver can be used to assert both 'satisfiability', i.e. that it is possible for the known values and the proposed value to be correct while following the defined rules, and 'validity', i.e. that the known values must result in the asserted value while following the defined rules. Roughly speaking, these equate to 'a' solution and 'the' solution. Obviously, 'the' solution is more useful; 'a' solution can be derived without setting any explicit values to other variables, but it won't tell us much.

Whitespace and newlines

For the ANTLR4 lexer and parser, a common approach is to skip all whitespace and newlines. This removes these elements from consideration, while still preventing tokenisation from taking place across text that is separated by either. Regardless, the sample benefit rules are keeping to roughly one expression per line, give or take occasional one-line bracketed expressions. The end of each statement is also followed by a newline, for readability and ease of structuring, but style is not enforced...

Comments

...except with comments! The language uses a `#` symbol to indicate the start of a comment, and the rest of the line until a newline is taken verbatim as the content of the comment, and none of it is matched to other code constructs.

Per Merigoux et al. (2022), the Catala DSL as a literate programming language extends commenting into a full annotation system that places the code implementation of each law in the context of an overall document structure and textual explanation of the rules that the code is designed to represent. That isn't part of the current design of the language, but tokenisation of comments is at least a step in the right direction, since it enables inclusion of comments into derived models. This could potentially, in future, form the basis of a textual explanation for proofs derived from the model.

However, tokenisation also requires that comments be explicitly recognised and assigned within each place in the code structure that they can appear. Per the Channels section of Tomassetti (2017), it's possible to direct comment tokens to a different channel with a lexer-only ANTLR4 configuration, thus letting them be used anywhere in a code file. Since the grammar as written is a combined lexer and parser, this option is not available. Rather than adding comment as a recognised token between every pair of elements, they've been relegated to only being recognised as complete statements, meaning they can be used before or after a full statement but not in the middle of one.

List

There is currently only one way that lists get used in Ben, and that is to create a new enum type. An enum type can have an arbitrary number of enum values, and we declare the type before we create a variable that can take one of the enum values defined in the type. A list is enclosed by brackets, with a comma between each list item.

For example:

```
Enum Housing_type(none, private, social, owner_occupier)
```

Expression

Expressions went through several stages of modelling by trial and error. The current thinking is that:

1. An expression does not stand alone in Ben. In rule definitions, statements only appear as part of function creation, where a variable is assigned to be equal to an expression. Expressions don't appear at all in test cases.
2. Brackets can be used to assert a non-default operator precedence in expression parsing, or for visual effect if the precedence is unchanged by their presence.
3. An expression can consist of two sub-expressions separated by an operator, or an if-then-else construct with three sub-expressions, or a single term which acts as the leaf node for its branch of an expression tree.
4. Each sub-expression follows the same rules as its parent, such that a single expression can grow into an expression tree as each of its sub-expressions has its own sub-expressions and so on.

This tree structure is why the visitor pattern is used to walk the parsed Ben file. Directly including the recursed calls to child nodes in Python expressions is easier if the whole statement can be evaluated at once.

In the language as defined, curly brackets are only used in a function declaration, that is, declaring a variable and asserting it as equal to the value produced by an expression. For example: `Money uc_earnings_taper = { (uc_net_earnings ~- uc_work_allowance) * 63% }`

Term

As mentioned above, terms are the leaf nodes in an expression tree. A term can be:

- An enum reference, i.e. a particular enum value such as `Housing_type.social`.
- A variable name, i.e. the name of a previously declared variable such as `uc_nondependent_deduction`. This could be a variable whose existence and type have been declared, or one that has been asserted as equal to the value of a function.
- A constant value, i.e. a fixed value of a defined type. Without using direct type-hinting, the form of each value must make it possible to lex, parse and

programmatically determine the value's type as well as the value's, er, value. For example, £34 is Money, and '2018-07-23' is Date.

All of these require careful definition and ordering of both lexer tokens and parser definitions.

Datatypes

The rules as described lend themselves to representation using strongly typed objects, since each rule always applies to the same type of value and produces the same type of output.

Using the sample benefit rules as a base, we need to handle integers (which are always non-negative in the sample rules), money, percentages, dates, Booleans, and enums. There is no place where we make explicit use of fractions or lists, although percentages are functionally fractions and enums are functionally a selection from a list of options.

In this section, we deal with representation in Ben. How these values are stored in Z3 and defined using Z3Py is discussed in the section for Objective 4.

Integer

Example: number of children is a non-negative integer used to determine whether a claimant comes under UC in rule 1, UC work allowance (rule 5), UC children's allowance (rule 8), childcare allowance (rule 11).

There do not appear to be any situations in the model where a negative number is carried forward from a calculation. For example, the earnings taper in rule 5 has a cut-off at 0, which I've implemented by taking the maximum of 0 or the calculated amount. There's an argument for creating a non-negative integer sort to ensure that Z3Py doesn't attempt to set negative values for variables when checking satisfiability and solvability.

Values of type integer will be just the numeric digits, e.g. 123, with no leading zeroes and the value zero represented by 0. Negative integer representation can be added in later, if necessary.

Money

All money in the sample rules is in pounds and pence, normally written like £123.45. For the purposes of this model, we assume that any multiplication or division of money results in another money value in pounds and pence, rounded to the nearest penny. This introduces a source of error, since by this approach, £3.01 / 4 is £0.75, but £0.75 * 4 is £3.00.

We need to support addition and subtraction (where both operands are money), and also multiplication (where one operand is an integer or a fraction) and division (where the denominator is an integer).

We define a variable as Money by writing `Money housing_costs_element`.

Values of type money are formatted as £1234.56, with no leading 0s. The pence element is optional, so e.g. £1234 is also valid.

Percent

Percentages are used in rule 5 and in the social landlord calculation of rule 12. In both cases, the function of the number is to take a percentage of a monetary value, thus returning another monetary value.

We define a percentage object by writing `Percent one_bedroom`.

To represent a percentage, we use the form `45%` where the number is an integer.

Date

Dates in benefit modelling follow the Gregorian calendar, hence following ISO 8601. They are used directly (e.g. a limit of the model is that date of claim must be between 5/3/2018 and 4/3/2019), and in more complex ways (age in years is a non-negative integer, calculated by comparing date of claim to date of birth following a set of rules).

We define a variable as Date by writing `Date date_of_birth`.

We will refer to dates in the standard ISO format YYYY-MM-DD, for example `'1977-03-13'`.

Boolean

Whenever eligibility is being determined, or a check is made as to whether something is the case or not, the required value is Boolean. For example, rule 1 determines whether a claimant comes under UC or not, so is either true or false.

We define a variable as Boolean by writing `Boolean is_under_UC`. To represent Boolean values, we use `true` and `false`.

Enum

There are rules that can place a claim in one of a list of situations, rather than just being true or false. In rule 12, housing costs could be one of ineligible or unclaimed, private tenant, registered social landlord tenant, or owner-occupier, each with its own eligibility requirements and calculation to determine the value of the housing costs element.

To represent Enum values, we define the possible values of an Enum variable, using e.g. `Enum housing_type(none, private, social, owner_occupier)`, and reference a specific value by writing `housing_type.private`.

Fraction - not implemented

There are places in the model where values are divided. It is possible for SMT solvers to handle values as fractions, which enables lossless comparison and presentation of results. Currently, there is no plan to deliberately incorporate that into modelling. Where division does occur, for example when calculating the core rent for private tenants in rule 12, the numerator is a money value, and the result is therefore a money value whose accuracy is limited to the nearest penny.

List or array - not implemented

The wider benefit system models eligibility for an arbitrary number of household members, including an arbitrary number of children. Eligibility and amounts vary both with the characteristics of each individual, and by their position in the overall list, ordered on some characteristic.

Operators

Assignment: =

There are two ways that assignment is used in the model being developed:

- `Money some_money = { money_a + money_b }` - assigning a variable to be equal to the value of an expression, essentially creating a function. In SMT-LIB2 this would be thought of more as asserting a condition on the model, since 'solving' the model is about trying to satisfy all the assertions. The code in every case is being written as `Datatype variable_name = { expression }`, complete with curly brackets.
- `some_variable = value` - assigning a value to one of the predefined variables. This is only used in the actual operation of models, to match the model to a test case or real world situation in order that an SMT solver can be used to derive the unknown values from the known ones, check satisfiability, or otherwise prove or disprove an assertion. So it should not be seen in the model code.

We could in theory define constants in the rules instead of directly including values in the rules base, and avoiding magic numbers is good practice generally. But for now, the rules use a deliberately limited subset of what's possible.

Equality: ==

Equality with two equals signs is a comparator operator, which returns true if the expressions on either side are equal, or false if they are not. As with other comparators, the expressions have predefined datatypes which must also be equal.

As a side note, I haven't followed the common shorthand of shortening the Boolean comparison `is_this_happening == true` to `is_this_happening` in my sample rules. There's no inherent reason why this shouldn't be supported though; an expression consisting of just a Boolean variable would pass through the value of that variable, which would be true or false.

Less-than and greater-than: < and >

Compare two operands of the same type, in the form `a < b`. The expression is true if the left-hand operand is less than the right-hand one, otherwise false. The expression `a > b` returns the same result as `b < a`.

Supported types include Date (for example, we know that in this model `date_of_claim < '2019-03-04'` is true), Integer. Both operands must be the same type.

Less-than-or-equal-to and greater-than-or-equal-to: `<=` and `>=`

Compare two operands of the same type, in the form `a <= b`. The expression is true if the left hand operand is equal to or less than the right-hand one, otherwise false. The expression `a <= b` is the same as `b >= a`.

Supported types include Date, Integer. Both operands must be the same type.

Addition: `+`

`a + b` returns the sum of the two operands. Both operands must be the same type. Supported types include Integer, Money.

Zero-bounded subtraction: `~-`

A noteworthy aspect of the current set of rules is that any expression involving subtraction treats a negative result as 0 (Integer) or £0 (Money). It's possible to support this by allowing negative values for Integer and Money types, then defining a `max(a, 0)` function to manually round up negative values to 0. I've decided instead to try replacing the standard minus operator with `~-`. This is the only operator that is substantially out of alignment with common operator definitions in other languages.

`a ~- b` returns the left-hand operand minus the right-hand operand if `b < a`, otherwise it returns 0 or £0. Both operands must be the same type and the result is the same type as the operands. Supported types include Integer, Money.

Multiplication: `*`

`a * b` returns the product of the two values. Supported types include Integer, Percent, Money, in the following combinations:

- Both operands are Integer, and the result is Integer.
- One operand is Money, the other is Integer, and the result is Money.
- One operand is Money, the other is Percent, and the result is Money. This implies that the result is rounded to the nearest penny.

Division: `/`

`a / b` is required for the private tenant calculation in rule 12, and returns the left-hand side divided by the right-hand side.

Minimum: `min`

Rule 11 has an upper bound for the childcare allowance, which can be represented with `a min £646.35` for one child, where `a` is a Money value representing 85% of the childcare cost and £646.35 is the maximum repayment for one child. It can also be thought of as `if (a < b) then a else b`.

Note that we're using an unconventional syntax for min by making it behave in the same way as our previous operators, with `min` in between its two expressions rather than placing it before and enclosing a list in brackets like `min(a, £646.35)`. We could go the conventional

route - it's been done this way for simplicity - but I'm holding off until or unless we want to compare lists of more than two items.

Supported types include Integer, Money, and potentially Date.

Conditional: if then else

A conditional expression takes the form `if a then b else c`, where `a` is Boolean, and `b` and `c` must have the same type as each other, which in turn is the same type as the return value of the expression. There's no reason to limit what that type is.

Conditional: case when when - not implemented

A form of `if then else` with an arbitrary number of comparisons, specifically adapted for ranges of values in this context. The author is aware of scenarios in the wider benefit system where the ability to split a range into categories (e.g. low, medium, high) based on comparison to cut-off points would substantially increase efficiency of modelling, but the current model does not have rules that would benefit from such a construct.

Maximum: max() - not implemented

As discussed earlier, `max(a, b)` would be needed to set a zero lower bound on subtraction if we didn't use zero-bounded subtraction. There don't appear to be any other uses in the present model, so there is no need to implement this operator at present.

Discussion points

Distinguishing eligibility and amount

One cause of uncertainty is that, for some calculations, it is unclear how best to communicate either (i) the amount of a benefit, or (ii) that the claimant is not eligible for the benefit in question. The options seem to be:

- If we use two variables, e.g. `ucEligible` and `ucAmount`, then it would be possible to represent impossible combinations, for example `ucEligible = false` and `ucAmount = £300`.
- If we use a combined variable, returning either the value or false, we could get meaningless calculations like `false - 50`.
- If we treat `ucAmount == £0` as meaning that a claimant is not eligible for UC, then we could theoretically find a failure mode where someone is eligible but their eligible amount is £0, and a further piece of logic that relies on eligibility status incorrectly assumes ineligibility.

The simplest way around this is to take care to follow the actual benefit logic when defining eligibility and amount of benefit. Where the two are calculated differently, such that a claimant could be eligible but for £0, we should define two separate functions, as per `ucEligible` and `ucAmount`. Where ineligibility is determined by a £0 amount, we can choose from just using `ucAmount`, or define `ucEligible = (ucAmount == £0)`. It should never be possible for the definitions to be so misaligned that `ucEligible` is false but `ucAmount` is non-zero.

Dealing with multiple instances of an 'object'

Another issue with the current language is that it doesn't support arrays. From personal experience, the author is aware that eligibility under the benefit rules at a point in time can be accurately modelled without use of arrays. However, it requires a significantly more complicated set of rules, with a substantial amount of duplication, and imposes an upper bound on collections that are not explicitly bounded, such as number of children, number of residents in a household, number of jobs being worked by each resident, and so on.

Objective 3: Create a parser to parse the benefit rule language

A standard way to create a language parser is to use a parser generator. Research shows that ANTLR is more productive than yacc-based parser generators (Ortin et al., 2022) when used by students. The current version of ANTLR is ANTLR4, and the grammar definition builds on extended Backus-Naur form (EBNF).

Accordingly, an ANTLR4 grammar was written for the Ben language, and a parser was generated from that. As this process was new to the author, there was considerable learning through trial and error. It was unclear how test-driven development could be used to drive the process, so a more interactive approach was taken, using the parse-tree visualisation utilities available in the Java-output version of the ANTLR4 parser generator to explore the impact of changes to the grammar and the rulesets, and to tweak the code until a satisfactory output was achieved.

Rather than a textbook, the primary resources used to understand and define an appropriate grammar were online, including Best Practices for ANTLR Parsers (Tomassetti, 2021) and The ANTLR Mega Tutorial (Tomassetti, 2017).

Once a grammar and Ben file were written that generated a correct parse-tree, the output destination was switched to Python 3, using guidance in the ANTLR4 README files (Parr, 2016) to generate a Python parser.

Most of the issues that arose during creation of the grammar interacted with issues involved in meeting other objectives, and are discussed in those. Operator precedence was an issue that most closely involved lexing and parsing, and is discussed below.

The grammar for Ben, in ANTLR4 format, can be found in [Appendix 3](#).

Operator precedence

While the grammar is simple, it is still desirable to be able to parse expressions such as `(age_of_claimant >= 16 and claimant_is_uc_eligible_young_person == true)`. In most languages, it is understood that this should be parsed as two sub-expressions, `age_of_claimant >= 16` and `claimant_is_uc_eligible_young_person == true`, which give Boolean results that are analysed using the `and` operator.

Since the language used for implementing parser and solver operation is Python, we shall use a slightly amended form of Python's order of operations (Python Software Foundation, 2022):

- Brackets
- Multiplication, division (*, /)
- Addition and bounded subtraction (+, ~-)
- Comparisons (==, <=, <, >=, >)
- Boolean AND (and)
- Boolean OR (or)
- min

The amendments are the inclusion of a 'min' operator with low precedence, and the exclusion of unsupported operators and curly brackets (which are supported in only one context). The method of establishing precedence, in both this and in token recognition such that e.g. Money or Date values are not parsed as integers and 'min' is not parsed as a variable name, is through use of the ANTLR4 grammar's first-come-first-serve ordering.

Objective 4: Create a runner that takes language code and sample data

For simplicity, Python was used to directly configure and run the SMT solver, via Z3Py, while parsing the Ben program. Thus, it becomes possible to write a Ben file, then use a single Python script to interpret and run it against an SMT solver without intervening steps or awkward, potentially dangerous creation of intermediate code files. This was an evolution of the original plan, which was to output SMT-LIB2 code to a file and then manually run that against an SMT solver.

Background reading that informed the development of the runner included Cok (2013), Bjørner et al. (2022), Hong (2020a, 2020b). The runner script can be found in [Appendix 4](#).

Listener vs visitor patterns

The default output from ANTLR4 to Python was a listener pattern. By experiment, it became clear that using Z3Py to directly define a solver object while walking through the parse tree using a listener poses issues. Z3Py methods expect well-formed statements with a beginning and end, because they are Python methods. Separate listener functions are called for entry to and exit from a parsed object, and for the children, children's children etc of each object.

It may be possible to use discrete Z3Py methods to create individual objects and connect each object to its parents as part of solver definitions, for example by declaring functions and then asserting equality between a symbol and the interpreted child node. However, listeners seemed more suited to outputting a new Python or SMT-LIB2 file, step by step, that would then need to be run separately in order to achieve the desired result.

It's possible to integrate direct Z3Py definition with a tree structure by using a visitor pattern instead of the listener, since it's trivial to nest calls to children within a Z3Py statement when

using a visitor. This is the approach that was taken, and the implementation is contained in the Python script at `src/ben_to_z3.py` in the GitHub repository.

Visitors are often discouraged as the stack size is unbounded, which can lead to stack overflow errors if a parsed expression is too complex. Python does not currently offer tail-call optimisation, and even if it did there would be difficulty in handling nodes with multiple child nodes, so there's no way to avoid adding to the stack with each child call. Running `sys.getrecursionlimit()` returns 1000 in the development environment, which is standard. For now, it seems sensible to operate within this imposed limit - functions can and should be broken up a very long time before the tree reaches 1000 levels deep.

Z3 representation of Ben datatypes

In Ben, all variables are in-scope, all the time. Consequently, we use two global objects:

- The solver `s` is declared as global, so that assertions can be made on it from any point in the tree-walking. Each function declaration is modelled as a Z3 assertion, and assertions are added to set boundaries on variables as they are declared.
- A global dict called `solver_objects` is used to store and access variables, with key equal to variable name and value equal to the variable.

The internal representations available to Z3 are rather more sparse than in a full language. For ease of modelling in the time available, it was decided to use a minimally feasible representation and logic for each datatype, rather than e.g. encoding leap years and date differences in the handling of dates.

There is a source of error present in the modelling of integer-based values in Z3. When an integer is divided, or multiplied by a percentage, the resulting value may have a remainder which is not captured when represented as an integer. This damages reversibility. For example, with a formula such as `big_value / 3 = little_value`, the solver may have a requirement to find a `little_value` that would cause `big_value` to be equal to 100, and be unable to solve as `33*3` is not 100, even though `100/3` gives 33 with a dropped remainder.

Integer

Integer has a direct equivalent in Z3, and using Z3Py we can define integer variables with `Int(var_name)` and integer values with `IntVar(value)`. Since we know that all integers have a zero lower bound, we can assert `s.add(solver_objects[var_name] >= 0)` to prevent the solver from satisfying the system of equations by setting values to less than zero.

Money

The simplest way of storing a Money value in Z3 is as an integer representing the total number of pence, i.e. `IntVar(pounds * 100) + pence`. As before, with the current model we can assert `s.add(solver_objects[var_name] >= 0)`.

Percent

Dealing with percentages is slightly more complex than money. While it's possible to create a fractional value using e.g. `IntVar(63) / IntVar(100)`, it's not possible to define a fractional variable as easily.

As it happens, percentages are only used four times in the current Ben file, each time as a fixed value multiplied by some other expression that equates to a Money value. No variables are defined as taking Percent values. This means that the Z3 solver does not need to handle fractional variables, and we can use the simple method defined before. However, if we do at some point want to use percentage variables then this will be revisited.

Date

In the sample rulesets, age is treated as an integer value that is entered separately from date of birth and date of claim, with no direct relationships between them. This means that the sample rulesets only use dates for the purpose of comparison, that is: less-than, greater-than, less-than-or-equal-to, greater-than-or-equal-to, and equality.

This is of particular importance because it removes the need to calculate differences in dates, which requires full date parsing and a set of rules for simple calculation of difference in years, and a different set of rules accounting for leap days when calculating difference in days. Difference in years would not be onerous to calculate (Compare yyyy, then mm, then dd), but it is still appreciably simpler to avoid the issue for now.

Because of this simplicity, we can represent dates in Z3 as strings, and modern Z3Py supports string comparison operators. The ISO standard format has the advantage that Z3Py string comparisons produce the same results as comparisons of the actual dates. So by directly storing the Ben date in ISO YYYY-mm-dd format as a `String`, unmodified, we can support date comparisons without having to build any new functions.

Boolean

The Ben values are true and false, and there is also a Boolean variable type to contend with. Since `Bool` is a Z3 type, we can just declare `Bool(var_name)`, and get a value using `BoolVal(ctx.getText().lower() == 'true')`.

Enum

Per Hong (2020a), it's possible to declare a custom enumeration type, then declare a variable that contains an enum of the declared type. Given that we want to declare enum types within the Ben code rather than imposing defined options in the parser, we need to support a multi-step process:

1. When the Ben file declares a custom enum type, create that enum type using Z3Py, store it in `solver_objects`.
2. When the Ben file declares a variable containing an enum of the custom type, create a variable using the custom type as its sort, and store it too in `solver_objects`.
3. When a term is an enum value, we get the enum corresponding to the enum's name, from the top-level enum type, using Python `getattr`.

Enums are barely used in the example Ben source, but will be crucial to any further extension of the benefit model.

The main function

When run as a Python script, the file at `src/ben_to_z3.py` in the GitHub repository parses a Ben file passed as an argument, walks the resulting abstract syntax tree, uses Z3Py to configure a Z3 solver according to the rules defined in the Ben file, populates variables with known values using a test case at the end of the Ben file, then attempts to determine the model's satisfiability and to solve the model.

The main function in the script builds on a template provided by Parr (2016), that is, the ANTLR4 Python README file. The `SmtLibConverter` class and functions override functions on the generic class, which were created programmatically in the `lib/` folder of the repository by ANTLR4 when generating a parser from the grammar that was defined in Objective 3.

Using the runner

Starting from project root, assuming that Python 3, Z3 and Z3Py are installed, the following will invoke the runner, parsing and executing the example Ben file:

```
cd src
python3 ben_to_z3.py SAMPLE_BENEFIT_RULES.ben
```

If no test case is included in the Ben file, then it should parse the model definitions, confirm that the model is satisfiable, i.e. that it's possible to find values for all of the variables that satisfy the rules that have been defined. It then drops to a Python REPL via `pdb`, for direct interaction with the model.

If a test case is included, i.e. there are `let` statements to set known values for variables and one or more `verify` statements to confirm the value of an output, then those are executed. The state of the model is changed by the `let` statements. The `verify` statement displays the validity of the proposed variable value or expression. Following these, it displays satisfiability and drops to the REPL.

Evaluation

The project has been evaluated on the following criteria:

1. The capabilities that have been achieved
2. The limitations of what's been achieved
3. Deviations from the original plan
4. What would have improved the process
5. Opportunities for further development and exploration

The capabilities that have been achieved

The project code repository, together with this project report, provide a functioning system consisting of:

- A clear, well-ordered, easily transcribed description of a subset of benefit rules to guide development of benefit models, along with test cases to demonstrate accuracy
- A fully-described language, Ben, which is sufficient to capture the chosen subset of benefit rules, and can be extended to meet further requirements
- An ANTLR4 grammar that lexes and parses code written in Ben into a well-formed abstract syntax tree
- A Ben code file that transcribes the chosen subset of benefit rules into a parseable form
- A Python script that uses the ANTLR4-generated parser to parse a Ben code file, then walks the resulting tree to create a Z3 representation of the Ben code using Z3Py

The instructions and documentation are sufficient to follow through each step of the process, and implement changes from one step to the next.

As-is, the system can be used to:

- Find or verify the value of missing variables in the benefit model, regardless of whether they're inputs or outputs.
- Test assertions made using Z3Py and the collection of variables and functions defined by the benefit model, and determine satisfiability and provability.
- Investigate the impact of changes made to individual elements of the model through manual overriding of values, either directly in interactive console or through repeated execution with altered Ben code.
- Output a 'proof' in Z3 format of a given assertion.

At a proof-of-concept level, the system therefore demonstrates the feasibility of all of the opportunities identified at the beginning of the project.

The limitations of what's been achieved

Most obviously, the interface and the usability of the Python script are very basic. This is an artefact of the development process, which led to the final script being written only after the delivery of all other steps. The capabilities described above require a knowledge of Python and Z3Py, a basic understanding of the model structure, and a willingness to write code in a Python REPL on the command line.

Related to this, the final script does not adhere to modern software quality standards.

While the basic use cases can be demonstrated, it is unclear whether some of the more ambitious goals of the system are achievable:

- SMT solvers can find 'a' value that satisfies the constraints placed on them, but determining a range of possible values appears to be non-trivial.
- Scalability is not proven in this context. Although Z3 can cope with the given benefit model, it is unclear how an SMT solver would cope with a ruleset consisting of a

hundred times the number of variables and assertions, all connected with each other. This would also apply to time-sensitive modelling with rules that changed by the time that they were applied. It would especially apply to batch processing. Large-scale modelling involves placing tens of thousands of sets of household data into a model in order to find missing values or determine the impact of changed conditions. Both speed and likelihood of finding a solution become more important in this use case, and neither has been measured at scale, especially not with a full-size benefit model.

- As discussed in the background, UK law builds on the common law system. Catala's examples are based in civil law. There may be a level of systemic complexity in UK law that results in every input and output having numerous caveats, making the trade-off between accuracy and ease of use rather more acute.
- Finally, the interface with probability theory is completely unexplored. Frankly, a probability-based approach seems like it would have very little overlap with one based on direct application of rules.

There are also some possibilities that arose during development, but were out of scope or unachievable in the time available:

- Finding a consistent way to integrate eligibility for and value of a benefit.
- Using list comprehensions to instantiate multiple objects. Although Z3 does support arrays, Hong (2020a) says that arrays 'should not be used to model small finite collections of values. It is usually much more efficient to create different variables using list comprehensions'.
- Using arrays to incorporate large collections of values into the model. There are elements to the benefit rules which depend on large numbers of key-value lookups. These are primarily geographical lookups, such as the Local Housing Allowance rate for a property or the Council Tax rate for a property.

Deviations from the original plan

The original plan was to proceed through one staged objective at a time, writing test cases, delivering the objective, and then moving to the next. This proved unrealistic.

First, all of the exciting problems were in the later stages, which resulted in unwillingness to call a halt on development at whatever stage had been reached in the time available.

Second, a lack of existing knowledge of the languages and technologies underlying some stages meant that the objectives were reached through a process of interaction with the technologies, and it was not clear what outputs could even be modelled using test cases before development was completed.

Third, the sheer quantity of work in achieving the objectives was overwhelmingly higher than anticipated.

Another part of the plan was to select random benefit rules from throughout the source book to provide a representative cross-section of the kinds of behaviour that needed to be modelled. It swiftly became clear that this would not meet most of the project aims, since the resulting models would consist of a single function or definition with no complexity or

interactions with other rules. Instead, a single, coherent set of rules was described and modelled.

What would have improved the process

The original plan was to deliver the system using a horizontal development strategy, developing one layer at a time, then moving to the next. In retrospect, this had extremely negative consequences for development. A vertical strategy is often recommended in software development, and this project was a lesson in why that's a good idea.

Taking a single rule, writing a one line Ben file, a rudimentary grammar, and a script to implement the single rule in PyZ3 would have taken less time than delivering a single horizontal slice. Instead of having to go back and revise each layer as part of designing the subsequent layer, it would be possible to see how everything fitted together and to design appropriately from the start. Finally, the quantity of rules being modelled could more easily be matched to the time available, instead of starting with a large set of rules and carrying them through in their entirety to the next layer.

Opportunities for further development and exploration

There are many paths forward:

- Extending the Ben language would make it possible to represent the wider set of benefit rules in a simple and coherent fashion.
- A complete set of test cases for each stage would provide greater verification of correctness.
- An up-to-date, complete benefit system model written in Ben would give evidence of scalability and make the model actually useful for end users.
- Improved UX / UI would make it possible for non-programmers to interact with the system, and make use of its capabilities.
- Further research and development would resolve some of the outstanding questions over what can be achieved.

Conclusion

The project objectives were partly met. While the sample set of benefit rules, the Ben language for expressing them, the grammar to parse files in that language, and the runner to convert Ben code into a Z3 model are all in place, both testing and UX / UI are lacking.

More importantly for the future, the project's aims were to explore and understand two opportunity spaces: modelling of benefits via a DSL, and integration of an SMT solver with a benefit model. and to provide a base for future development in the field of benefit modelling.

The author's belief is that both of these spaces have been explored, resulting in clear evidence of future opportunity. Further, with the creation and documentation of a proof of concept that encompasses the entire modelling process from rules description to SMT solving, a base has been laid for future development.

While many of the opportunities identified at the beginning have either been delivered or been proven to be deliverable, there are a number of outstanding questions that would need further research to determine. These, too, would benefit from building on the structures created in delivery of Ben. To that extent, the author regards the project as a success.

References and Credits

In general, system design decisions are based on previous, personal experience of good practice and design pitfalls, with reasoning laid out in this project report and / or in comments to the code. External sources of design logic included the project brief and example program code and blog posts sourced online.

Where program code or algorithms are substantially based on the work of others, I acknowledge this in comments above the code and in the citations section below. Where code follows a standard 'boilerplate' structure without room for deviation or creativity, this is not acknowledged, as it is common to all code that makes use of the platforms in question.

Bibliography

Administrative Data Research UK (2022). *Administrative Data*. [online] ADR UK. Available at: <https://www.adruk.org/our-mission/administrative-data/> [Accessed 2 Oct. 2022].

Bjørner, N., de Moura, L., Nachmanson, L. and Wintersteiger, C. (2022). *Programming Z3*. [online] theory.stanford.edu. Available at: <https://theory.stanford.edu/~nikolaj/programmingz3.html> [Accessed 2 Oct. 2022].

Child Poverty Action Group (2018). *Welfare benefits and tax credits handbook*. 20th ed. London: Child Poverty Action Group.

Cok, D. (2013). *The SMT-LIBv2 Language and Tools: A Tutorial*. [online] Available at: <https://smtlib.github.io/jSMTLIB/SMTLIBTutorial.pdf> [Accessed 2 Oct. 2022].

Delaët, A., Merigoux, D. and Fromherz, A. (2022). Turning Catala into a Proof Platform for the Law. [online] POPL 2022 - Programming Languages and the Law. Available at: <https://hal.inria.fr/hal-03447072/document> [Accessed 2 Oct. 2022].

Department for Work and Pensions (2016). *Universal Credit transition to full service*. [online] GOV.UK. Available at: <https://www.gov.uk/government/publications/universal-credit-transition-to-full-service> [Accessed 2 Oct. 2022].

Directorate of Legal and Administrative Information (2022). *Simulateur de calcul pour 2022 : impôt sur les revenus de 2021 (Simulateur)*. [online] www.service-public.fr. Available at: <https://www.service-public.fr/particuliers/vosdroits/R2740> [Accessed 2 Oct. 2022].

Hong, E. (2020a). *Z3Py Advanced*. [online] ericpony.github.io. Available at: <https://ericpony.github.io/z3py-tutorial/advanced-examples.htm> [Accessed 2 Oct. 2022].

- Hong, E. (2020b). *Z3Py Guide*. [online] [ericpony.github.io](https://ericpony.github.io/z3py-tutorial/guide-examples.htm). Available at: <https://ericpony.github.io/z3py-tutorial/guide-examples.htm> [Accessed 2 Oct. 2022].
- Husa, J. (2016). *The Future of Legal Families*. [online] *Oxford Handbooks Online*. Oxford University Press. doi:10.1093/oxfordhb/9780199935352.013.26.
- Mackley, A. (2022). *Social security powers in the UK. Briefing Paper no. 9048*. [online] London: House of Commons, pp.4–5. Available at: <https://researchbriefings.files.parliament.uk/documents/CBP-9048/CBP-9048.pdf> [Accessed 2 Oct. 2022].
- Merigoux, D., Chataing, N., Rolley, E., Gesbert, L., Fromherz, A., Delaët-Tixeuil, A. and Slimani, L. (2022). *catala/README.md at master · CatalaLang / catala*. [online] GitHub. Available at: <https://github.com/CatalaLang/catala#concepts> [Accessed 2 Oct. 2022].
- Merigoux, D., Monat, R. and Protzenko, J. (2021). A Modern Compiler for the French Tax Code. In: *Proceedings of the 30th ACM SIGPLAN International Conference on Compiler Construction (2021): n. pag.* [online] Available at: <https://arxiv.org/abs/2011.07966> [Accessed 2 Oct. 2022].
- Ortin, F., Quiroga, J., Rodriguez-Prieto, O. and Garcia, M. (2022). An empirical evaluation of Lex/Yacc and ANTLR parser generation tools. *PLoS ONE*, [online] 17(3), pp.e0264326–e0264326. doi:10.1371/journal.pone.0264326.
- Parr, T. (2016). *antlr4/doc/python-target.md at 4.6 · antlr/antlr4*. [online] GitHub. Available at: <https://github.com/antlr/antlr4/blob/4.6/doc/python-target.md> [Accessed 2 Oct. 2022].
- Python Software Foundation (2022). 6. *Expressions — Python 3.10.7 documentation*. [online] docs.python.org. Available at: <https://docs.python.org/3/reference/expressions.html#operator-precedence> [Accessed 2 Oct. 2022].
- Richiardi, M., Collardo, D. and Popova, D. (2021). UKMOD – A new tax-benefit model for the four nations of the UK. *International Journal of Microsimulation*, [online] 14(1), pp.92–101. doi:10.34196/ijm.00231.
- The National Archives (2018). *Understanding Legislation*. [online] [Legislation.gov.uk](https://www.legislation.gov.uk). Available at: <https://www.legislation.gov.uk/understanding-legislation#Howlegislationworks> [Accessed 2 Oct. 2022].

Tomassetti, G. (2017). *The ANTLR Mega Tutorial*. [online] Strumenta. Available at: <https://tomassetti.me/antlr-mega-tutorial/> [Accessed 2 Oct. 2022].

Tomassetti, G. (2021). *Best Practices for ANTLR Parsers*. [online] Strumenta. Available at: <https://tomassetti.me/best-practices-for-antlr-parsers/> [Accessed 2 Oct. 2022].

UK Government (2022). *Benefits calculators*. [online] GOV.UK. Available at: <https://www.gov.uk/benefits-calculators> [Accessed 2 Oct. 2022].

Appendices

Appendix 1: Sample benefit rules, written as a Ben file

The following rules were originally in the same order as the rulesets in Objective 1, but were reordered during development because our language requires that a variable or function must be declared before it is used in an expression. The rulesets in Objective 1 took the opposite approach, of defining a parent rule, then all of its children.

```
# This is a datatype more than a variable
Enum Housing_type(none, private, social, owner_occupier)

# We declare variables before invoking them
# Some of these can be converted to functions later
# There are unmodelled dependencies contained in these variables
# For example, age_of_claimant determines claimant_is_below_pension_age
Boolean rolled_out_to_full_service
Date date_of_claim
Integer number_of_children
Boolean is_uc_eligible_young_person
Boolean is_couple
Integer age_of_claimant
Boolean claimant_is_uc_eligible_young_person
Boolean claimant_is_below_pension_age
Boolean claimant_not_in_education_or_exempt
Boolean claimant_is_resident_in_uk
Boolean claimant_right_to_reside
Boolean claimant_is_in_great_britain
Boolean claimant_is_subject_to_immigration_control
Boolean claimant_has_accepted_claimant_commitment
Integer age_of_partner
Boolean partner_is_uc_eligible_young_person
Boolean partner_is_below_pension_age
Boolean partner_not_in_education_or_exempt
Boolean partner_is_resident_in_uk
Boolean partner_right_to_reside
Boolean partner_is_in_great_britain
Boolean partner_is_subject_to_immigration_control
Boolean partner_has_accepted_partner_commitment
Money claimant_net_earnings
Money partner_net_earnings
Date dob_of_eldest_child
Integer number_of_children_with_lower_rate
Integer number_of_children_with_higher_rate
Boolean claimant_has_limited_capability_for_work_related_activity
Boolean partner_has_limited_capability_for_work_related_activity
Integer number_of_carees
```

Money childcare_spend
 Housing_type uc_housing_type # i.e. uc_housing_type holds an enum from Housing_type
 Integer number_of_relevant_family_occupants
 Integer number_of_liable_occupants
 Money local_housing_allowance_for_your_family
 Integer number_of_nondependant_family_occupants
 Money rent
 Integer uc_housing_excess_bedrooms
 Money combined_savings
 Money uc_unearned_income # This is ruleset 6, oddly

```

# Ruleset 1 - When you come under the universal credit system
# In practice, anyone for whom this is false falls outside of the model
# Don't include logic for UC Gateway claimants as they're outside of model dates
Boolean comes_under_uc = {
  (rolled_out_to_full_service == true)
  and (date_of_claim >= '2018-05-03')
  and (date_of_claim <= '2019-04-03')
  and (number_of_children < 3)
}

```

```

# Ruleset 2 - Are you eligible for Universal Credit?
# We've reversed order of definitions because our solver requires
# variables to be defined *before* we use them
Boolean claimant_eligible_for_uc = {
  ((age_of_claimant >= 18) or (age_of_claimant >= 16 and
  claimant_is_uc_eligible_young_person == true))
  and (claimant_is_below_pension_age == true)
  and (claimant_not_in_education_or_exempt == true)
  and (claimant_is_resident_in_uk == true and claimant_right_to_reside == true)
  and (claimant_is_in_great_britain == true)
  and (claimant_is_subject_to_immigration_control == false)
  and (claimant_has_accepted_claimant_commitment == true)
}

```

```

Boolean partner_eligible_for_uc = {
  ((age_of_partner >= 18) or (age_of_partner >= 16 and
  partner_is_uc_eligible_young_person == true))
  and (partner_is_below_pension_age == true)
  and (partner_not_in_education_or_exempt == true)
  and (partner_is_resident_in_uk == true and partner_right_to_reside == true)
  and (partner_is_in_great_britain == true)
  and (partner_is_subject_to_immigration_control == false)
  and (partner_has_accepted_partner_commitment == true)
}

```

Ruleset 12 - A housing costs element under UC

The qualifying period makes £0 the default value for new claims

```
Money uc_housing_owner_occupier_cost = {  
    £0  
}
```

```
Money uc_housing_bedroom_tax = {  
    if uc_housing_excess_bedrooms == 0  
    then £0  
    else (  
        if uc_housing_excess_bedrooms == 1  
        then rent * 14%  
        else rent * 25%  
    )  
}
```

```
Money uc_housing_social_cost = {  
    rent ~- uc_housing_bedroom_tax  
}
```

```
Money uc_nondependent_deduction = {  
    £72.16 * number_of_nondependant_family_occupants  
}
```

```
Money uc_housing_private_cap_rent = {  
    local_housing_allowance_for_your_family  
}
```

```
Money uc_housing_private_core_rent = {  
    (rent * number_of_relevant_family_occupants) / number_of_liable_occupants  
}
```

```
Money uc_housing_private_cost = {  
    (uc_housing_private_core_rent min uc_housing_private_cap_rent) ~-  
    uc_nondependent_deduction  
}
```

```
Money uc_housing_costs_element = {  
    if uc_housing_type == Housing_type.none  
    then £0  
    else (  
        if uc_housing_type == Housing_type.private  
        then uc_housing_private_cost  
        else (  
            if uc_housing_type == Housing_type.social  
            then uc_housing_social_cost  
            else uc_housing_owner_occupier_cost  
        )  
    )  
}
```

```

)
}

# Ruleset 5 - What is the earnings taper?
# We've reversed order of definitions because our solver requires
# variables to be defined *before* we use them
Money uc_net_earnings = {
  claimant_net_earnings + partner_net_earnings
}

Money uc_work_allowance = {
  if (number_of_children > 0) or (claimant_has_limited_capability_for_work_related_activity
== true or partner_has_limited_capability_for_work_related_activity == true)
  then (
    if uc_housing_costs_element > 0
    then £198
    else £409
  )
  else £0
}

Money uc_earnings_taper = {
  (uc_net_earnings ~- uc_work_allowance) * 63%
}

# Ruleset 7 - The standard UC allowance
Money uc_standard_allowance = {
  if (is_couple == true)
  then (if (age_of_claimant < 25 and age_of_partner < 25) then £395.20 else £498.89)
  else (if age_of_claimant < 25 then £251.77 else £317.82)
}

# Ruleset 8 - An allowance for children under UC
Money uc_children_allowance = {
  if number_of_children == 0
  then £0
  else ((
    (if dob_of_eldest_child < '2017-04-06'
    then £277.08
    else £231.67)
    +
    (if number_of_children == 2
    then £231.67
    else £0)
  )
  + (number_of_children_with_lower_rate * £126.11)
  + (number_of_children_with_higher_rate * £383.86))
}

```

Ruleset 9 - An element for adults with limited capability for work or work-related activity due to illness under UC

```
Money uc_work_incapability_element = {  
  if (claimant_has_limited_capability_for_work_related_activity == true  
    or partner_has_limited_capability_for_work_related_activity == true)  
  then £328.32  
  else £0  
}
```

Ruleset 10 - A carer element if one of the recipients cares for a severely disabled person under UC

```
Money uc_carer_element = {  
  if uc_work_incapability_element > £0  
  then £0  
  else (  
    if number_of_carees >= 1  
    then (  
      £156.45  
      + (  
        if (number_of_carees >= 2 and is_couple == true)  
        then £156.45  
        else £0  
      )  
    )  
    else £0  
  )  
}
```

Ruleset 11 - An allowance for childcare under UC

We assume that all children are under 16

```
Boolean is_uc_childcare_eligible = {  
  if number_of_children == 0  
  then false  
  else (  
    if is_couple == true  
    then (  
      if claimant_net_earnings > £0  
      then (  
        if (partner_net_earnings > £0 or  
partner_has_limited_capability_for_work_related_activity)  
        then true  
        else false  
      )  
    )  
    else (  
      if (claimant_has_limited_capability_for_work_related_activity and partner_net_earnings  
> £0)  
    )  
  )  
}
```

```

        then true
        else false
    )
)
else (
    if claimant_net_earnings > £0
    then true
    else false
)
)
}

```

```

Money uc_childcare_proportion = {
    childcare_spend * 85%
}

```

```

Money uc_childcare_element = {
    if is_uc_childcare_eligible == true
    then (
        if number_of_children == 1
        then (uc_childcare_proportion min £646.35)
        else (uc_childcare_proportion min £1108.04)
    )
    else £0
}

```

```

# Ruleset 4 - How much is your maximum amount of UC?
# We've reversed order of definitions because our solver requires
# variables to be defined *before* we use them
Money uc_maximum_amount = {
    uc_standard_allowance
    + uc_children_allowance
    + uc_work_incapability_element
    + uc_carer_element
    + uc_childcare_element
    + uc_housing_costs_element
}

```

```

# Ruleset 3 - How much Universal Credit do you get?
# Note the use of ~- means this cannot go below £0
Money uc_entitlement = {
    uc_maximum_amount ~- uc_earnings_taper ~- uc_unearned_income
}

```

```

# Carried forward from Ruleset 2
# We've reversed order of definitions because our solver requires
# variables to be defined *before* we use them
Boolean eligible_for_uc = {

```

```
(comes_under_uc == true)
and (claimant_eligible_for_uc == true)
and ((is_couple == false) or (partner_eligible_for_uc == true))
and (combined_savings <= £16000)
and (uc_entitlement > £0)
}
```

Appendix 2: Sample test cases, as Ben files

Each test case is run by pasting it at the end of the existing Ben file that defines the benefits model.

Test Case 1

let rolled_out_to_full_service = true

let date_of_claim = '2018-05-03' # Juuuust in the valid period

let is_couple = true

let age_of_claimant = 26

let age_of_partner = 26

let number_of_children = 2

let dob_of_eldest_child = '2017-04-06' # Juuuust in the valid period

let claimant_net_earnings = £1

let partner_net_earnings = £0

let partner_has_limited_capability_for_work_related_activity = true # Is this correct?

let uc_housing_type = Housing_type.none

let uc_unearned_income = £0

verify uc_maximum_amount == £1290.55

Fun fact: this produces a disproof, because it's possible to reach £1290.56!

Appendix 3: Ben lexer and parser, written as an ANTLR4 grammar file

```
grammar BENEFIT_LANGUAGE;

/*
 * Parser
 * We do not enforce type consistency in the parser. This is possible to do
 * as there are very few types and operators, but complex validation is best
 * left to the visitor.
 */

// Top-level structure
// from Tomassetti (2021)
file
: statements EOF
;

statements
: (statement | test_statement)+
;

statement
: (comment | declare_function | declare_variable | declare_enum_type |
declare_enum_variable)
;

test_statement
: (assign_variable | verify_value)
;

/*
 * Test file / solver statements
 */

// Set a variable equal to a value
assign_variable
: LET VARIABLE_NAME '=' (value | enum_reference)
;

// Verify that a variable is equal to a value
verify_value
: 'verify' VARIABLE_NAME '==' (value | enum_reference)
;

/*
 * Ben model statements
```

```

*/

declare_function
: declare_variable '=' '{ expression }'
;

declare_enum_variable
: ENUM_VARIABLE_NAME VARIABLE_NAME
;

declare_variable
: VARIABLE_TYPE VARIABLE_NAME
;

declare_enum_type
: 'Enum' ENUM_VARIABLE_NAME OPEN_BRACKET VARIABLE_NAME
(LIST_SEPARATOR VARIABLE_NAME)* CLOSE_BRACKET
;

expression
: '(' unbracket=expression ')'
| left=expression multdiv=('*'|'/') right=expression
| left=expression plusminus=('+'|'~') right=expression
| left=expression comparison=('=='|'<='|'<'>='|'>') right=expression
| left=expression and='and' right=expression
| left=expression or='or' right=expression
| left=expression min=MIN right=expression
| ite=if_then_else
| atom=term
;

if_then_else
: 'if' expression 'then' expression 'else' expression
;

term
: enum_reference | VARIABLE_NAME | value
;

value
: PERCENT | MONEY | DATE | INTEGER | BOOLEAN
;

enum_reference
: ENUM_VARIABLE_NAME '.' VARIABLE_NAME
;

// Everything from a # to end of line is marked as a comment

```

```

// Note that this DOES NOT WORK if a comment is placed inside a statement
// To do that well, we'd need to either manually add comment matches at all
// points in every statement, or use channels which are not available in
// combined lexer / parser
comment
: COMMENT
;

/*
 * Lexer
 */

PERCENT
: INTEGER '%'
;

MONEY
: CURRENCY_SYMBOL INTEGER (DECIMAL_POINT PENCE_VALUE)?
;

DATE
: SINGLE_QUOTE YEAR '-' MONTH '-' DAY SINGLE_QUOTE
;

INTEGER
: NON_ZERO_DIGIT DIGIT*
| '0'+
;

BOOLEAN
: 'true'
| 'false'
;

YEAR
: NON_ZERO_DIGIT DIGIT DIGIT DIGIT
;

MONTH
: DIGIT DIGIT
;

DAY
: DIGIT DIGIT
;

// Punctuation and structure

```

LIST_SEPARATOR

: ','
;

OPEN_CURLY_BRACKET

: '{'
;

CLOSE_CURLY_BRACKET

: '}'
;

OPEN_BRACKET

: '('
;

CLOSE_BRACKET

: ')'
;

COMMENT

: HASH_SYMBOL REST_OF_LINE
;

LET // Above VARIABLE_NAME

: 'let'
;

MIN // This operator goes above VARIABLE_NAME to establish precedence

: 'min'
;

// Data types

VARIABLE_TYPE // Excludes Enum as that is defined at time of declaration

: 'Integer' | 'Money' | 'Percent' | 'Date' | 'Boolean'
;

ENUM_VARIABLE_NAME

: UPPER_CASE_LETTER (LOWER_CASE_LETTER | UNDERSCORE)*
;

VARIABLE_NAME

: (LOWER_CASE_LETTER | UNDERSCORE)+
;

WHITESPACE

: [\t]+ -> skip

```

;

NEWLINE
: ('\r' | '\n')+ -> skip
;

// from Tomassetti (2021)
ANY
: .
;

// Fragments

fragment SINGLE_QUOTE
: '\"
;

fragment PENCE_VALUE
: DIGIT DIGIT
;

fragment DECIMAL_POINT
: '.'
;

fragment NON_ZERO_DIGIT
: [1-9]
;

fragment DIGIT
: '0' | NON_ZERO_DIGIT
;

fragment CURRENCY_SYMBOL
: '£'
;

fragment UNDERSCORE
: '_'
;

fragment UPPER_CASE_LETTER
: [A-Z]
;

fragment LOWER_CASE_LETTER
: [a-z]
;

```

```
fragment HASH_SYMBOL
: '#'
;
```

```
fragment REST_OF_LINE
: ANY*? NEWLINE
;
```

Appendix 4: Python and Z3Py runner script

```
# Based on Parr (2016)
import sys
from pathlib import Path
from z3 import *
import pdb

# Add lib to system path so we can import the generated files
# They're kept in a separate directory to prevent mixing of generated
# and written code. This path setting fails if the execution
# directory is different from the script location in src
lib_path = Path("../lib/")
sys.path.append(str(lib_path)) # TODO: replace this with __init__.py ?
from antlr4 import *
from BENEFIT_LANGUAGELexer import BENEFIT_LANGUAGELexer
from BENEFIT_LANGUAGEParser import BENEFIT_LANGUAGEParser
from BENEFIT_LANGUAGEVisitor import BENEFIT_LANGUAGEVisitor

# Holds declared Z3 objects, e.g. variables, in a global namespace
solver_objects = {}
s = Solver()

def main(argv):
    """
    Takes a BEN file as input, outputs to SMT-LIB2 with same filename,
    different extension
    """
    global s
    program = FileStream(fileName=argv[1], encoding='utf-8')
    lexer = BENEFIT_LANGUAGELexer(program)
    stream = CommonTokenStream(lexer)
    parser = BENEFIT_LANGUAGEParser(stream)
    tree = parser.file_()
    smt_lib_converter = SmtLibConverter()
    print("Begun solver setup")
    output = smt_lib_converter.visit(tree)
    print("Completed solver setup")
    if s.check() == sat:
        print("\nThe system is satisfiable.")
    else:
        print("\nThe system is not satisfiable.")
    # Drop to REPL. The output is present if wanted
    pdb.set_trace()

class SmtLibConverter(BENEFIT_LANGUAGEVisitor):
```

```
'''
```

```
Each node either visits its children, or processes the content
of itself and its immediate children if it's at the lowest
level. We use globals throughout, partly because we want to
define objects in statements and then access them in later,
non-children statements, partly because the Ben language
has no namespaces, at present.
```

```
'''
```

```
# Helper functions
```

```
def Min(self, a, b):
    return If(a < b, a, b)
```

```
def Max(self, a, b):
    return If(a > b, a, b)
```

```
# Visit a parse tree produced by BENEFIT_LANGUAGEParser#file.
```

```
def visitFile(self, ctx: BENEFIT_LANGUAGEParser.FileContext):
    return self.visitChildren(ctx)
```

```
# Visit a parse tree produced by BENEFIT_LANGUAGEParser#statements.
```

```
def visitStatements(self, ctx: BENEFIT_LANGUAGEParser.StatementsContext):
    return self.visitChildren(ctx)
```

```
# Visit a parse tree produced by BENEFIT_LANGUAGEParser#statement.
```

```
def visitStatement(self, ctx: BENEFIT_LANGUAGEParser.StatementContext):
    return self.visitChildren(ctx)
```

```
# Visit a parse tree produced by BENEFIT_LANGUAGEParser#test_statement.
```

```
def visitTest_statement(self, ctx: BENEFIT_LANGUAGEParser.Test_statementContext):
    return self.visitChildren(ctx)
```

```
# Visit a parse tree produced by BENEFIT_LANGUAGEParser#assign_variable.
```

```
def visitAssign_variable(self, ctx: BENEFIT_LANGUAGEParser.Assign_variableContext):
```

```
'''
```

```
    Let a variable = a value, as part of test case setup
```

```
'''
```

```
    global s
```

```
    global solver_objects
```

```
    var_name = ctx.VARIABLE_NAME().getText()
```

```
    if ctx.value() is not None:
```

```
        var_value = self.visit(ctx.value())
```

```
    else:
```

```
        var_value = self.visit(ctx.enum_reference())
```

```
    print(solver_objects[var_name], " == ", var_value)
```

```
    s.add(solver_objects[var_name] == var_value)
```

```
# Visit a parse tree produced by BENEFIT_LANGUAGEParser#verify_value.
```

```
def visitVerify_value(self, ctx: BENEFIT_LANGUAGEParser.Verify_valueContext):
```

```

'''
Verify that a variable == a value, i.e. must be that value, to confirm test case output
'''

global s
global solver_objects
var_name = ctx.VARIABLE_NAME().getText()
if ctx.value() is not None:
    var_value = self.visit(ctx.value())
else:
    var_value = self.visit(ctx.enum_reference())

if s.check() == sat:
    print("\nThe system is satisfiable.")
else:
    print("\nThe system is not satisfiable.")
print("\nResult of validity check:")
print(prove(solver_objects[var_name] == var_value))

# Visit a parse tree produced by BENEFIT_LANGUAGEParser#declare_function.
def visitDeclare_function(self, ctx:
BENEFIT_LANGUAGEParser.Declare_functionContext):
    global s
    declared_var = self.visit(ctx.getChild(0))
    inside_brackets = self.visit(ctx.getChild(3))
    s.add(declared_var == inside_brackets)

# Visit a parse tree produced by BENEFIT_LANGUAGEParser#declare_enum_variable.
def visitDeclare_enum_variable(self, ctx:
BENEFIT_LANGUAGEParser.Declare_enum_variableContext):
    global solver_objects
    enum_name = ctx.children[0].getText()
    var_name = ctx.children[1].getText()
    # This declares a constant rather than variable, which is
    # not strictly incorrect - it's an unknown constant
    solver_objects[var_name] = Const(var_name, solver_objects[enum_name])

# Visit a parse tree produced by BENEFIT_LANGUAGEParser#declare_variable.
def visitDeclare_variable(self, ctx:
BENEFIT_LANGUAGEParser.Declare_variableContext):
    global solver_objects
    global s
    var_type = ctx.getChild(0).getText()
    var_name = ctx.getChild(1).getText()
    # There is currently no way of carrying out type checking
    # TODO: Create and use custom types to ensure type safety?
    if var_type == "Integer":
        solver_objects[var_name] = Int(var_name)
        # To force non-negative integer, we assert a constraint on values taken

```

```

        s.add(solver_objects[var_name] >= 0)
    elif var_type == "Money":
        solver_objects[var_name] = Int(var_name)
    elif var_type == "Percent":
        # Not currently needed, and would require a fair amount of
        # implementation logic to handle well
        raise NotImplementedError("Unsupported behaviour: Percent variables not currently
supported.")
    elif var_type == "Date":
        solver_objects[var_name] = String(var_name)
    else:
        solver_objects[var_name] = Bool(var_name)
    # We return the newly created variable, in case it forms part of a
    # function declaration, and the parent wants to use what we've
    # declared to make an assertion, in SMT-LIB-speak
    return solver_objects[var_name]

# Visit a parse tree produced by BENEFIT_LANGUAGEParser#declare_enum_type.
def visitDeclare_enum_type(self, ctx:
BENEFIT_LANGUAGEParser.Declare_enum_typeContext):
    # Using a global dictionary as a central namespace
    # for storing Z3 variables, constants etc
    global solver_objects
    enum_name = ctx.children[1].getText()
    solver_objects[enum_name] = Datatype(enum_name)
    number_of_enum_values = (ctx.getChildCount() - 3) // 2
    for x in range(number_of_enum_values):
        # Note this is 0 to number_of_enum_values -1
        solver_objects[enum_name].declare(ctx.children[(x * 2) + 3].getText())
    solver_objects[enum_name] = solver_objects[enum_name].create()

# Visit a parse tree produced by BENEFIT_LANGUAGEParser#expression.
def visitExpression(self, ctx: BENEFIT_LANGUAGEParser.ExpressionContext):
    if ctx.left is not None:
        left = self.visit(ctx.left)

    if ctx.right is not None:
        right = self.visit(ctx.right)

    if ctx.unbracket is not None:
        return self.visit(ctx.unbracket)

    if ctx.multdiv is not None:
        if ctx.multdiv.text == "**":
            return left * right
        else:
            return left / right

```

```

if ctx.plusminus is not None:
    if ctx.plusminus.text == "+":
        return left + right
    else: # ctx.plusminus.text == "~-"
        # Add a zero-lower bound, the manual way
        return self.Max(left - right, Int(0))

if ctx.comparison is not None:
    op_text = ctx.comparison.text
    if op_text == "==":
        return left == right
    elif op_text == "<=":
        return left <= right
    elif op_text == "<":
        return left < right
    elif op_text == ">=":
        return left >= right
    else: # op_text == ">"
        return left > right

if ctx.and_ is not None:
    return And(left, right)

if ctx.or_ is not None:
    return Or(left, right)

if ctx.min_ is not None:
    return self.Min(left, right)

if ctx.ite is not None:
    return self.visit(ctx.ite)

if ctx.atom is not None:
    return self.visit(ctx.atom)

raise ValueError("Somehow, an expression doesn't match any of the possible
expression tokenisations")

# Visit a parse tree produced by BENEFIT_LANGUAGEParser#if_then_else.
def visitIf_then_else(self, ctx: BENEFIT_LANGUAGEParser.If_then_elseContext):
    # In the current definition, if, then, else are elements 0, 2, 4
    return If(self.visit(ctx.children[1]), self.visit(ctx.children[3]), self.visit(ctx.children[5]))

# Visit a parse tree produced by BENEFIT_LANGUAGEParser#term.
def visitTerm(self, ctx: BENEFIT_LANGUAGEParser.TermContext):
    global solver_objects
    # A term is either a value, or the name of a variable, or an enum reference
    if ctx.value() is not None:

```

```

        return self.visitChildren(ctx) # Does this return an object or an array?
    elif ctx.VARIABLE_NAME() is not None:
        var_name = ctx.getText()
        return solver_objects[var_name]
    else: # ctx.enum_reference() is not None
        return self.visit(ctx.enum_reference())

# Visit a parse tree produced by BENEFIT_LANGUAGEParser#value.
def visitValue(self, ctx: BENEFIT_LANGUAGEParser.ValueContext):
    """
    Parses a value and returns it as a Z3Py value. This reimplements
    the lexer splitting of text into different elements. I can't
    find a way to bring the individual tokens into the parser without
    breaking lexing, which relies on having the full combination of
    tokens present in the ordering to ensure that a value is
    matched and tokenised appropriately.
    """
    # TODO: This also could use some type safety
    if ctx.PERCENT() is not None:
        # Treating x% as x / 100, both Z3 ints
        return IntVal(ctx.getText()[:-1]) / IntVal(100)
    elif ctx.MONEY() is not None:
        # Treat money as an IntVal of total in pence
        money_array = ctx.MONEY().getText()[1:].split(".")
        money_val = int(money_array[0]) * 100
        if len(money_array) == 2:
            money_val += int(money_array[1])
        return IntVal(money_val)
    elif ctx.DATE() is not None:
        # We save the full date as a string value in Z3Py,
        # because Z3Py supports < <= > >= on strings nowadays
        return StringVal(ctx.DATE().getText().strip(""))
    elif ctx.INTEGER() is not None:
        return IntVal(ctx.getText())
    else: # ctx.BOOLEAN() is not None
        # Note that lower() is strictly unnecessary; True doesn't parse
        return BoolVal(ctx.getText().lower() == 'true')

# Visit a parse tree produced by BENEFIT_LANGUAGEParser#enum_reference.
def visitEnum_reference(self, ctx:
BENEFIT_LANGUAGEParser.Enum_referenceContext):
    enum_var = solver_objects[ctx.ENUM_VARIABLE_NAME().getText()]
    enum_attribute = ctx.VARIABLE_NAME().getText()
    return getattr(enum_var, enum_attribute)

# Visit a parse tree produced by BENEFIT_LANGUAGEParser#comment.
def visitComment(self, ctx: BENEFIT_LANGUAGEParser.CommentContext):
    # We return nothing

```

```
# print(ctx.getText())  
pass
```

```
if __name__ == '__main__':  
    main(sys.argv)
```